

Capstone Project Briefs

14 Projects · Student Edition

This document contains the official project briefs for the capstone component of the Automation with Generative AI course. Each brief defines exactly what you are expected to build, what must be included, what your README must document, and what tech stack to use.

Projects are grouped into three tracks. You choose one project. Two students may choose the same project — your implementation is individual and will be compared against the same brief.

How you will be reviewed

Your submission is reviewed by Claude AI against this brief, and then by a QAE reviewer who tests your live URL. Claude checks whether your implementation covers every Must Include point. The QAE reviewer checks whether it works correctly and whether the output is complete and useful.

What to submit

Public GitHub repository URL

README.md covering all points listed in README Requirements for your chosen project

Live deployed URL (Render recommended for Streamlit projects)

Project 01

Document Q&A;

Upload any document and ask questions — get grounded answers with source references.

WHAT YOU ARE BUILDING

Build a Streamlit app where a user uploads a PDF and asks questions about its content. The app chunks the document, retrieves relevant sections, and sends them to Claude to generate a grounded answer. The app must show which section of the document the answer came from. If the answer is not in the document, Claude must say so rather than guessing.

MUST INCLUDE — ALL REQUIRED

- Accept PDF upload up to 20 pages
- Document chunked before sending to AI — not sent as one block
- Answer displayed with source section reference (page number or chunk)
- If answer not found in document, AI responds 'not found in document' explicitly
- Download full Q&A; session as formatted PDF report
- Handles corrupted or empty PDF gracefully with a user-facing message
- API key from environment variable

README MUST DOCUMENT

- Chunking strategy used and chunk size
- How source references are tracked and displayed
- Limitations — max pages supported, file size limit

RECOMMENDED STACK

[Claude API](#) · [Streamlit](#) · [pypdf](#) · [python-dotenv](#)

Project 02

Resume Screener

Upload a job description and multiple CVs — get a ranked shortlist with reasoning.

WHAT YOU ARE BUILDING

Build a Streamlit app where a hiring manager uploads a job description and multiple resume PDFs. The AI reads each resume, scores it against the JD on defined criteria, and ranks all candidates. Output shows ranked list with score breakdown per candidate. Export ranked results as CSV.

MUST INCLUDE — ALL REQUIRED

- Accept one JD file and multiple resume PDFs (minimum 3 resumes)
- Scoring criteria defined in the prompt — at least 4 criteria (skills match, experience, education, other)
- Each resume scored per criterion with a total score
- Ranked list displayed in the app from highest to lowest
- Export ranked results as CSV with columns: rank, name, total score, score per criterion, reasoning
- Handles resumes with non-standard formatting gracefully

- API key from environment variable

README MUST DOCUMENT

- Scoring criteria and how they are weighted
- How candidate name is extracted from resume
- Limitations — AI scoring is advisory, not a hiring decision

RECOMMENDED STACK

[Claude API](#) · [Streamlit](#) · [pypdf](#) · [python-dotenv](#)

Project 03

Invoice Processor

Upload an invoice image or PDF — get a structured data table you can export.

WHAT YOU ARE BUILDING

Build a Streamlit app where a user uploads an invoice as a PDF or image. The AI extracts vendor name, invoice number, date, line items with quantities and prices, subtotal, tax, and total. Results displayed as a structured table in the app. Export as Excel file.

MUST INCLUDE — ALL REQUIRED

- Accept PDF and image upload (JPG, PNG)
- Extract at minimum: vendor name, invoice number, date, all line items, subtotal, tax, total
- Display extracted data as a clean structured table
- Export as Excel file with proper column headers
- If a field is not found, show 'not found' rather than leaving blank silently
- Handles multi-page invoices
- API key from environment variable

README MUST DOCUMENT

- Which AI model is used and why (vision capability required)
- Extraction fields and how missing fields are handled
- Excel export format and column names

RECOMMENDED STACK

[GPT-4o API \(vision\)](#) · [Streamlit](#) · [python-dotenv](#) · [openpyxl](#)

Project 04

Multi-Model Comparator

Same prompt, three models, one side-by-side view.

WHAT YOU ARE BUILDING

Build a Streamlit app where a user types any prompt and sends it to Claude, GPT-4o, and Gemini simultaneously. All three responses displayed side by side. User can rate each response. Export the full comparison including prompt, all responses, latency, and user ratings as PDF.

MUST INCLUDE — ALL REQUIRED

- All three models called — Claude, GPT-4o, Gemini

- Responses displayed side by side in columns
- Response latency shown per model in milliseconds
- User can rate each response 1-5 stars
- Export comparison as PDF: prompt, responses, latency, ratings
- If one model fails, other two still display with an error note for the failed one
- API keys for all three from environment variables

README MUST DOCUMENT

- How to configure three separate API keys
- How parallel or sequential calls are handled
- Latency measurement approach

RECOMMENDED STACK

[Claude API](#) · [OpenAI API](#) · [Gemini API](#) · [Streamlit](#) · [python-dotenv](#)

Project 05

Content Repurposer

Paste one article — get LinkedIn post, tweet thread, and email newsletter ready to publish.

WHAT YOU ARE BUILDING

Build a Streamlit app where a user pastes an article or blog post. The AI generates three outputs: a LinkedIn post (under 1300 characters), a Twitter/X thread (5-7 tweets numbered), and an email newsletter version (subject line + body with intro, main points, CTA). User picks target audience. Download all three as a formatted Word document.

MUST INCLUDE — ALL REQUIRED

- Accept article text paste
- Target audience selector that measurably changes tone and vocabulary
- Three distinct outputs: LinkedIn post, Twitter thread, email newsletter
- LinkedIn post under 1300 characters — show character count
- Twitter thread numbered correctly (1/7, 2/7 etc)
- Email newsletter includes subject line, intro, main points, call to action
- Download all three as a single formatted document
- API key from environment variable

README MUST DOCUMENT

- How target audience affects the prompt
- Character limit enforcement for LinkedIn
- Document format and how the three sections are structured

RECOMMENDED STACK

[Claude API](#) · [Streamlit](#) · [python-dotenv](#)

Project 06

Meeting Notes Processor

Paste a meeting transcript — get action items, decisions, and a summary in under a minute.

WHAT YOU ARE BUILDING

Build a Streamlit app where a user pastes a meeting transcript. The AI extracts action items with assigned owners and deadlines, key decisions made, open questions, and a 3-line executive summary. Results displayed in structured sections. Export as formatted PDF or CSV of action items only.

MUST INCLUDE — ALL REQUIRED

- Accept transcript text paste
- Extract four distinct sections: action items, decisions, open questions, summary
- Action items include owner name and deadline where mentioned
- 3-line summary is concise and accurate
- Export full structured output as PDF
- Export action items only as CSV: item, owner, deadline
- Handles transcripts with no clear owner names gracefully
- API key from environment variable

README MUST DOCUMENT

- How owner and deadline extraction works in the prompt
- What happens when owners or deadlines are not mentioned in the transcript
- CSV format for action items export

RECOMMENDED STACK

[Claude API](#) · [Streamlit](#) · [python-dotenv](#)

Project 07

Structured Data Extractor

Paste any unstructured text — define your fields — get clean JSON or CSV out.

WHAT YOU ARE BUILDING

Build a Streamlit app where a user pastes any unstructured text (complaint emails, form responses, news articles) and defines the fields they want to extract (e.g. name, product, issue, urgency). The AI extracts those exact fields from the text. Batch mode: upload a CSV of multiple texts, extract fields for all, download results.

MUST INCLUDE — ALL REQUIRED

- User defines extraction fields in the app — not hardcoded
- Single mode: one text input, extracted fields displayed as table
- Batch mode: CSV upload with one text per row, extract from all
- Output shows extracted JSON in the app and allows CSV download
- If a field is not found in the text, output 'not found' for that field
- Batch download CSV has original row plus one column per extracted field
- API key from environment variable

README MUST DOCUMENT

- How user-defined fields are passed to the prompt

- Batch CSV format — required column name for the text column
- Limitations — extraction accuracy depends on how clearly fields are named

RECOMMENDED STACK

[Claude API](#) · [Streamlit](#) · [python-dotenv](#)

Track 2 — Automation Focused · n8n as the Core

Project 08

Lead Qualification Pipeline

A form submission becomes a scored lead in Sheets with a Slack alert — automatically.

WHAT YOU ARE BUILDING

Build an n8n workflow triggered by Google Form submission. The form collects name, email, company, company size, and what they need. n8n sends the data to AI which scores the lead 0-100 with reasoning and classifies as hot/warm/cold. Results written to Google Sheets. Slack alert sent to the sales channel with lead summary and score. Different Slack message format for hot vs warm vs cold leads.

MUST INCLUDE — ALL REQUIRED

- Google Form collects at minimum: name, email, company, company size, requirement
- n8n triggers automatically on submission
- AI produces lead score 0-100 with reasoning and hot/warm/cold classification
- Google Sheet row written with all form data plus score, classification, timestamp
- Slack alert sent to sales channel with name, company, score, classification
- Different Slack message colour or format for hot vs warm vs cold
- If AI call fails, lead still written to Sheets with score marked as 'pending'
- API key in n8n credentials

README MUST DOCUMENT

- n8n workflow node diagram
- Lead scoring criteria in the prompt
- Google Sheet column structure
- Slack message format for each lead classification

RECOMMENDED STACK

[n8n](#) · [Claude API](#) · [Google Forms](#) · [Google Sheets](#) · [Slack](#)

Project 09

Email Classifier And Router

Every incoming email automatically classified, routed, and logged — no manual triage.

WHAT YOU ARE BUILDING

Build an n8n workflow triggered by incoming email. AI classifies the email into one of four categories: support, sales, complaint, or internal. n8n routes a summary to the correct Slack channel per category and logs a structured entry to Google Sheets. Urgency level (high/medium/low) also extracted and

included in the Slack message.

MUST INCLUDE — ALL REQUIRED

- Email trigger in n8n — Gmail or webhook
- AI classifies into exactly: support, sales, complaint, internal
- AI extracts urgency level: high, medium, low with brief reason
- Different Slack channel per category — all four must be configured
- Slack message includes: sender, subject, category, urgency, one-line summary
- Google Sheet logs: timestamp, sender, subject, category, urgency, summary
- Unclassified or ambiguous emails routed to a fifth 'review' channel
- API key in n8n credentials

README MUST DOCUMENT

- n8n workflow structure and how the routing branches work
- Classification prompt and how ambiguous cases are handled
- Slack channel naming and setup instructions
- Google Sheet column structure

RECOMMENDED STACK

[n8n](#) · [Claude API](#) · [Gmail](#) · [Google Sheets](#) · [Slack](#)

Project 10

Content Approval Workflow

AI drafts content, team approves in Slack, it gets logged automatically.

WHAT YOU ARE BUILDING

Build an n8n workflow where a user submits a content brief via Google Form (topic, tone, platform, word count). n8n sends to AI which drafts the content. Draft posted to a Slack channel with approve and reject buttons (emoji reactions). On approval reaction, content logged to Google Sheets with status Approved and publish date. On rejection, logged with status Rejected and a prompt for feedback.

MUST INCLUDE — ALL REQUIRED

- Google Form collects: topic, tone, platform, target word count
- n8n triggers automatically, calls AI, posts draft to Slack
- Slack message shows the draft clearly with brief and word count
- Approval emoji triggers: log to Sheets with Approved status + timestamp
- Rejection emoji triggers: log to Sheets with Rejected + prompt feedback message in Slack
- Google Sheet columns: topic, platform, status, approved by, timestamp, content
- Word count of generated content shown in Slack message
- API key in n8n credentials

README MUST DOCUMENT

- Which emojis trigger approval and rejection and how to configure them
- n8n workflow diagram
- Google Sheet structure

- How tone and platform affect the AI prompt

RECOMMENDED STACK

[n8n](#) · [Claude API](#) · [Google Forms](#) · [Google Sheets](#) · [Slack](#)

Project 11

Job Application Tracker

Submit a CV and job description — get a fit score, structured profile, and hiring manager notification automatically.

WHAT YOU ARE BUILDING

Build an n8n workflow triggered by Google Form. Applicant submits name, email, CV text, and job description. AI extracts structured candidate profile (skills, experience, education) and scores fit against the JD. Results written to Google Sheets. Hiring manager notified via email with candidate summary and fit score. Applicant receives confirmation email.

MUST INCLUDE — ALL REQUIRED

- Google Form collects: name, email, CV text, job description text
- n8n triggers automatically on submission
- AI extracts structured profile: top 5 skills, years of experience, education level
- AI produces fit score 0-100 with top 3 reasons for the score
- Google Sheet row: name, email, fit score, skills, experience, education, timestamp
- Hiring manager email sent with candidate summary and score
- Applicant confirmation email sent with application received message
- If AI call fails, both parties emailed with notification of manual review needed

README MUST DOCUMENT

- n8n workflow diagram
- Profile extraction fields and prompt structure
- Fit scoring criteria
- Email templates used for applicant and hiring manager

RECOMMENDED STACK

[n8n](#) · [Claude API](#) · [Google Forms](#) · [Google Sheets](#) · [Gmail](#)

Track 3 — Combined · n8n + Python + Streamlit

Project 12

Rag Chatbot

Ingest your documents via a webhook, query them through a chat interface.

WHAT YOU ARE BUILDING

Build a system where n8n ingests documents via webhook, chunks and embeds them into ChromaDB. A Streamlit chat interface lets users ask questions. Claude retrieves relevant chunks and answers with citations. Chat history maintained within the session. Export chat history as text file.

MUST INCLUDE — ALL REQUIRED

- n8n webhook accepts document text and triggers chunking and embedding
- OpenAI embeddings API used for embedding chunks into ChromaDB
- Streamlit shows a chat-style interface with message history
- Each AI answer cites the source chunk used
- If answer not in documents, AI says so explicitly
- Chat history persists within the session (not lost on each message)
- Export full chat history as text file
- API keys from environment variables

README MUST DOCUMENT

- Architecture diagram: n8n + ChromaDB + Streamlit
- How to run both n8n and Streamlit locally
- Chunking approach
- Citation format in answers

RECOMMENDED STACK

n8n · **Claude API** · **OpenAI Embeddings** · **ChromaDB** · **Streamlit** · **python-dotenv**

Project 13

Customer Support Dashboard

Support tickets come in via webhook, AI classifies and drafts responses, dashboard shows live queue.

WHAT YOU ARE BUILDING

Build a system where n8n receives support tickets via webhook, AI classifies by category and urgency and drafts an initial response. A Streamlit dashboard shows the live ticket queue with status, category, urgency, and AI draft per ticket. QA team can mark tickets as resolved in the dashboard.

MUST INCLUDE — ALL REQUIRED

- n8n webhook receives ticket: customer name, email, issue text
- AI classifies: category (billing/technical/general) and urgency (high/medium/low)
- AI drafts a professional initial response to the customer
- All tickets stored persistently — visible in dashboard after restart
- Streamlit dashboard shows: ticket ID, customer, category, urgency, status, AI draft
- Filter by category and urgency in the dashboard
- Mark as resolved button changes ticket status
- API key from environment variable

README MUST DOCUMENT

- Architecture diagram: n8n webhook and Streamlit app
- How tickets are stored between sessions
- Dashboard filter options
- How to send a test ticket to the webhook

RECOMMENDED STACK

Project 14

Research Aggregator

Feed URLs into n8n, get a consolidated AI-generated research report in Streamlit.

WHAT YOU ARE BUILDING

Build a system where n8n fetches content from multiple URLs, sends each to the AI for summarisation, and extracts key points and source credibility assessment. A Streamlit dashboard displays the consolidated research report with all sources. User can add more URLs. Export full report as PDF.

MUST INCLUDE — ALL REQUIRED

- n8n accepts URLs via webhook or manual trigger
- Each URL fetched and content extracted
- AI produces per source: summary (under 150 words), 3 key points, credibility note
- All source summaries consolidated into one research report in Streamlit
- New URLs can be added without losing previous research
- Export consolidated report as PDF with all sources cited
- If a URL fails to load, show error for that source and continue with others
- API key from environment variable

README MUST DOCUMENT

- n8n workflow structure
- How to add new URLs to an existing research session
- PDF export format and citation style
- Limitations — scraping may fail for JS-heavy or paywalled sites

RECOMMENDED STACK

n8n · Claude API · Streamlit · python-dotenv
